

从 DeepSeek Harness 看 Agent 自进化

让 Agent 自己改进自己，最大的障碍往往不是「不知道怎么改」，而是「改了之后出问题怎么办」。DeepSeek Harness（下文简称 dsh）用一套「插件化 + 可回滚」的架构，把修改自己的运行环境变成一件可控、可撤销、可验证的事。本文先讲清楚自进化的困难与 DeepSeek 的基本判断，再看它的插件化与热插拔是如何做到「不污染上下文、不打断运行」的，最后结合 Self-Harness 与 ACE 两篇论文，说明底座之上完整的自进化路线。

[插件化 · 可逆效应 · 事件日志](#)[Cordis 论文精读](#)[Self-Harness · ACE](#)[面向初学者](#)

01 Agent 自进化难在哪里

所谓 Agent 自进化，指的是 Agent 改进自己的行为——调整提示词、增删工具、修改流程，乃至修改模型本身。这个方向听起来自然，但无论走哪条路，都会撞上具体的困难。可以从数据、离线、在线三个层面来看。

1.1 数据层面：判断「改动有没有用」，本身就需要好数据

自进化的每一步都要回答同一个问题：这次改动到底带来了什么影响？变好了、变坏了，还是只是波动？要可靠地回答它，需要两样东西：**量级够大的任务集**（在几十个任务上的「提升」很可能只是噪声）和**质量够高的任务分布**（任务要能代表真实使用场景，否则优化的方向本身就是错的）。

其中关键的一环是 **query（任务请求）的筛选**：哪些任务用来发现问题、哪些用来验证改进、哪些应该排除（太简单、太难、或者答案已经泄漏给模型）。筛选不当，自进化就会退化成「针对练习题刷分」——指标在涨，真实能力没变。这一层数据门槛，**离线自进化和在线自进化都绕不开**。

1.2 离线自进化：两条主路都不轻松

离线场景（部署之前批量优化）有两条主要路径，各有各的难处：

训练路线（微调模型）

用 Agent 的执行轨迹去训练模型。困难在于：**成本高**（大规模算力与数据）；**流程长**（收集轨迹→标注→训练→评估，一轮下来周期以周计）；**Reward 难生效**（Agent 任务的奖励信号稀疏、延迟，还容易被「钻空子」——模型学会迎合评分标准，而不是真正把任务做好）；整体路径不清晰。

分析路线（不训练，归纳经验）

从执行轨迹中归纳出改进——改提示词、改工具说明、改流程规则。成本低、见效快。但归纳出的修改**很难达到最优解**：经验有限、归纳有偏差；更难**保证每次改动都是正向优化**：改动之间可能此消彼长，按下葫芦浮起瓢。

一个值得观察的中间方向

把 Agent 的脚手架配置**参数化**——提示词、工具说明、流程参数统一编码成可优化的向量——在向量空间中用训练或搜索的方法优化，再**解码**回具体配置。相当于「训练的是配置而不是权重」：有望把训练方法的系统性，与分析方法的低成本结合起来。这个思路目前还是一个开放问题，值得持续关注。

1.3 在线自进化：最终目标，核心是可控性

在线场景——Agent 一边服务用户、一边持续改进自己——是自进化的最终形态。它在前两层困难之外，又多出一类全新的难题：**怎么保证自我修改是可控的**。具体是三件事：

① 改坏了怎么恢复

Agent 给自己打了「补丁」，结果新补丁有缺陷。如果系统没有记录改动之前的状态，就没有退路——像编辑文档却没存历史版本。

② 改动会不会影响别人

组件相互依赖：工具 A 依赖文件系统，文件系统上还挂着一堆监听器。运行中替换一个组件，其他部分能否正常协作？

③ 改动会不会干扰正在运行的任务

Agent 正在处理一个长任务，此时替换组件，会不会弄丢已经积累的对话和中间结果？——类似「给飞行中的飞机换发动机」。

这正是 DeepSeek Harness 正在探索的问题——用插件化、可回滚、日志隔离来回应这三个可控性难题（第 3、4 节展开）。需要说明的是，**这套方案目前也并不完美**，仍在快速演进之中；它给出的是一个有工程说服力的方向，而不是终点。

本节要点

自进化的困难分三层：**数据层**——判断改动效果需要量大质优的任务与筛选机制（第 5、6 节两篇论文的任务集设计是对它的回应）；**离线层**——训练路线贵而慢，分析路线难保最优（第 5 节 Self-Harness 与第 6 节 ACE 是分析路线的两个代表，前者管「怎么改」，后者管「改的素材从哪来」）；**在线层**——可控性，第 3、4 节的 DeepSeek Harness 底座回应它。后文按这条线索展开。

02 DeepSeek 对自进化的理解

在展开 dsh 之前，先简单认识一下它背后的公司，以及这项工作在 DeepSeek 整体版图中的位置。

DeepSeek · 背景

DeepSeek（深度求索）是一家 2023 年 7 月成立于杭州的人工智能公司。它先后开源了 DeepSeek-V2、V3 等大语言模型；2025 年 1 月发布的推理模型 R1，以明显更低的训练成本达到了当时第一梯队的推理能力，并采用 MIT 协议完全开源，在全球范围内引发了广泛关注——开发者可以自由地研究、部署和二次开发它的模型，公开的技术报告也把训练方法讲得很透。这种「开源 + 公开细节」的风格延续到了 Agent 方向：

DeepSeek Harness (dsh) 同样是开源项目，其背后的编程范式研究整理为 Cordis 论文（《A Programming Paradigm for Spatiotemporal Composability》，北京大学 + DeepSeek-AI 联合工作）。

基于 Cordis 论文的工作，DeepSeek 对自进化有一个很清晰的基本判断。

2.1 自进化不一定要改模型权重

一个 Agent 的表现，并不完全由模型权重决定。**系统提示词怎么写、有哪些工具可用、工具说明是否清楚、执行流程怎么安排、结果怎么验证**——这些「模型之外」的配置，同样深刻影响着 Agent 的能力。这一层通常被称为 **harness**（脚手架 / 运行环境）。

修改脚手架，比修改模型权重有三个现实优势：**成本低**（不需要重新训练）、**可解释**（改了什麼一目了然）、**可撤销**（配置文件可以退回上一个版本）。所以 DeepSeek 选择的路线是：**把自进化的问题，转化为「如何安全地修改脚手架」的问题**。

CORDIS 论文的预见

Cordis 论文在动机部分 (§1.2.2) 明确讨论过「自进化的 Agent 脚手架」这一场景：未来的运行环境可能需要在服务请求的同时，**生成并部署对自身组件的修改**。要做到这一点，系统必须同时满足两个条件——改动可以完整撤销（时间维度），组件之间能感知并适应彼此的变化（空间维度）。这两个条件正是 Cordis 全部设计的目标。

2.2 但前提是：脚手架本身必须「可安全修改」

问题在于，传统 Agent 框架的脚手架是一团「硬编码」：工具列表写死在代码里，提示词散落在各处，组件之间在启动时绑定。想在运行中修改它，就回到了第 1 节的三个困难。

所以 DeepSeek Harness 的定位不是「又一个 Agent 框架」，而是一个为「**被修改**」而设计的运行底座。它把整个 Agent 拆成可以单独替换的零件，并为每次替换提供登记、撤销和验证的机制。下图概括了「底座」与「自进化闭环」的分工关系：

底座与闭环 · 自进化的两层结构

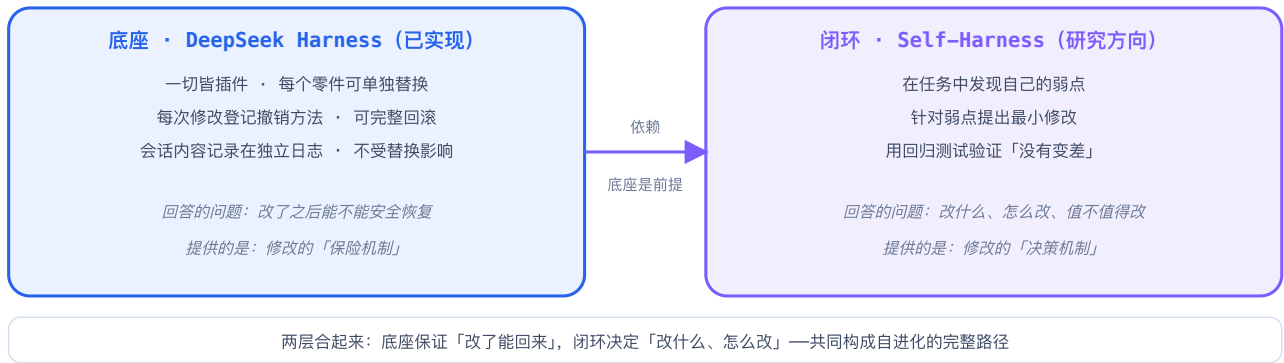


图 2-1 · 自进化的两层结构: 安全底座 (dsh) + 决策闭环 (Self-Harness)

03 DeepSeek Harness 的三个精髓

dsh 的仓库有 200 多个软件包, 细节繁多。但对理解自进化而言, 抓住三个核心设计就够了——它们分别回应第 1 节 1.3 中在线可控性的三个难题。

一切皆插件

模型调用、工具执行、文件系统、界面、甚至 Agent 的主循环本身, 全部是插件。没有哪个部件拥有特权地位, 任何零件都可以按名字替换、关闭或新增。**结构上, 修改自己 = 替换插件。**

每次修改都登记撤销方法

系统里修改状态只有一个入口: `ctx.effect()`。每次调用它做修改时, 必须同时登记一个「撤销函数」(相当于存档点)。插件被卸载时, 系统自动执行这些撤销函数, 把环境恢复原状。**解决了「改坏了怎么恢复」。**

会话内容只记录、不篡改

Agent 与用户的所有交互(消息、工具调用、返回结果)都写入一本「只追加、不修改」的日志——像流水账本, 写上就不能擦。模型看到的一切内容, 都能从这本日志重建出来。**解决了「改动干扰运行中的任务」。**

DeepSeek Harness = 一棵可替换的插件树 + 一个唯一的修改入口 + 一本不可篡改的会话日志。
新行为通过挂新插件实现；旧行为通过卸载插件撤销；正在进行的会话由日志独立承载，不依赖任何具体插件。

3.1 事件机制：组件之间怎么协作

插件之间不直接调用彼此，而是通过「事件」协作。dsh 里有四种事件分发方式，其中最常用的一种在源码里叫 `waterfall`（字面意思是「瀑布」）。名字听起来抽象，机制其实很常见——它就是我们熟悉的中间件接力：

- 每个事件像一条**接力跑道**，跑道上按顺序站着若干个插件；
- 参数从第一个传到最后一个，每个插件拿到参数后可以选择**原样传下去、修改后传下去，或者直接打断**（不再往后传）；
- 最后一个插件处理完，结果就是这条事件的最终输出。

这个设计让「干预执行流程」变成了一件很平常的事：想给所有工具调用加一层权限检查？在跑道上插入一个新插件就行，不需要改动任何现有代码。其他三种分发方式（`parallel` 多个插件同时收到的广播、`serial` 按顺序逐个等待、`emit` 纯通知）各有适用场景，但思路一致：**组件通过挂事件来参与系统，而不是写死在流程里。**

04 插件化与热插拔的原理

这一节回答两个具体的问题：替换插件时，为什么不会污染上下文？为什么不会影响正在运行的任务？

4.1 怎么不污染上下文：登记—撤销机制

先解释「污染上下文」是什么意思：一个插件安装时，会往运行环境里添加服务、注册监听器、修改配置——这些都是它留下的「痕迹」。如果卸载时不清理干净，残留的痕迹就会影响后续运行，这就是污染。

dsh 的解法是**让痕迹必然可清除**。规则很简单：



关键在撤销的顺序：**后进的先出**（LIFO，Last In First Out）。就像从一摞盘子里拿盘子，最后放上去的最先被拿走。之所以必须这样，是因为后来的修改可能建立在早前修改的基础上——只有先撤销后面的，再撤销前面的，每一步才都能正确还原。

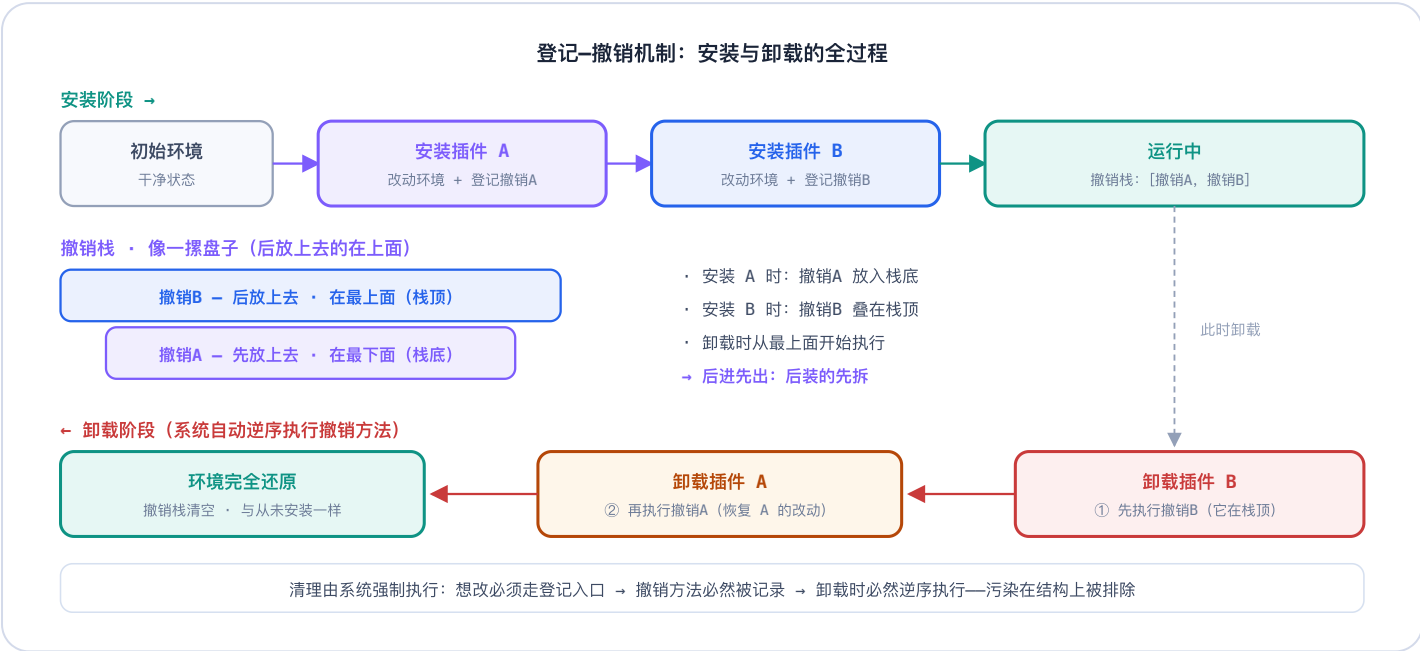


图 4-1 · 登记-撤销机制：每次修改都存一个「存档点」，卸载时逆序还原

为什么这样就不会污染

因为清理不是靠插件「自觉」，而是**系统强制的**：想改环境，必须走唯一的修改入口；走了入口，撤销方法就必然被登记；卸载时，系统必然逆序执行它们。污染的可能性在结构上被排除了，而不是靠开发者小心。

4.2 怎么不影响运行：状态与组件分离

第二个问题更微妙：替换组件时，Agent 可能正在处理一个长任务，上下文里已经积累了大量对话和工具结果。这些「正在进行的状态」放在哪里，决定了替换会不会弄丢它们。

dsh 的答案是**把状态和组件彻底分开**：

- **会话状态（对话历史、工具结果）全部写在日志里**。日志是独立的一层，只追加、不修改，不隶属于任何插件；
- **插件只提供「处理能力」，不持有「会话数据」**。模型调用走插件 A 还是插件 B，日志本身没有任何变化；
- 需要给模型发请求时，系统从日志**重新推导**出该发的内容——推导过程是确定性的，同样的日志必然得到同样的请求。

带来的效果是：热替换（在程序不重启的情况下更换零件）可以按下面四步安全进行——

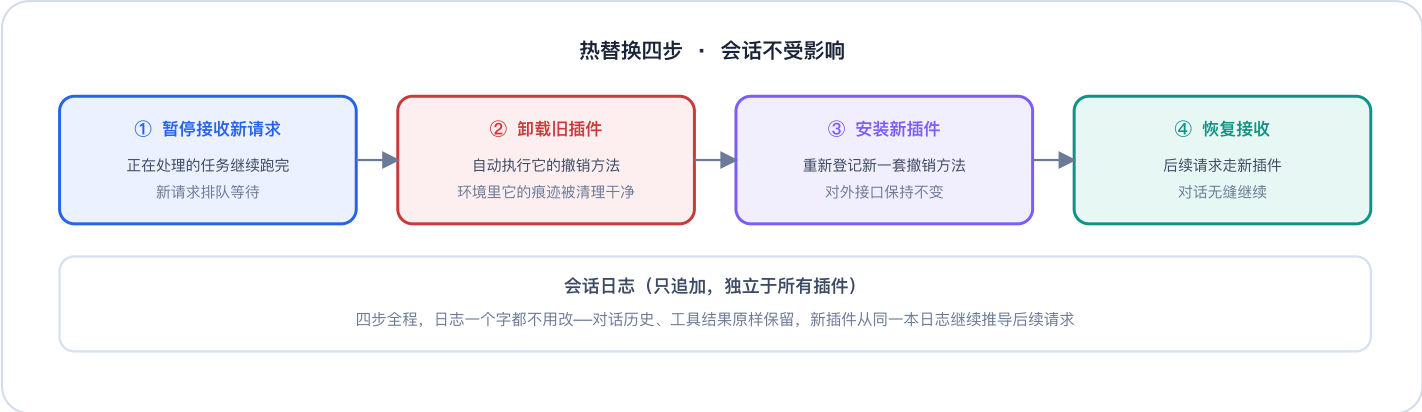


图 4-2 · 热替换流程：组件在换，日志不动——所以正在进行的会话不受影响

换个角度理解

可以把 dsh 想象成一家餐厅：插件是厨师，日志是账本。换厨师（热替换插件）时，账本一页都不用撕——新厨师照着同一本账本，就能接着服务这一桌客人。只要「事实记录」和「处理能力」是分开的，换零件就不会丢状态。

4.3 可替换带来的自由度

在这套机制下，「换零件」变得很日常。一个有代表性的例子：dsh 的命令行执行有三种实现——本机直接执行、沙箱隔离执行、远程虚拟机执行。三种实现的接口完全相同，替换时只改一个配置项，模型看到的工具名称和使用方式不变，Agent 的其他部分也不需要任何改动。

这正是自进化需要的自由度：改进 Agent 的某个方面，可以精确到「只换那一个零件」，而不必重写整个系统。改动范围小，意味着风险小、验证容易、出问题时空回滚也快——这些性质加在一起，才让「Agent 自己动手改自己」从危险操作变成可控操作。

05 自进化闭环：从底座到自动改进

底座解决了「改了能安全恢复」，但还有一半问题没回答：Agent 应该改什么？怎么知道改了之后真的变好了？Self-Harness 论文（arXiv:2606.09498，上海人工智能实验室）补上了这一半。

5.1 实验设定：只改脚手架，不改模型

Self-Harness 做了一个很干净的实验：把模型权重完全冻结（选了 MiniMax M2.5、Qwen3.5、GLM-5 三个基座，参数一个不动），只允许修改脚手架的配置——系统提示词、工具说明、验证规则。然后让

Agent 自己完成「发现弱点 → 提出修改 → 验证效果」的完整循环。

为了公平地评估改进是否真实，实验用了两套互不重叠的任务集：

- **练习任务集**：用来发现问题、总结弱点的任务。Agent 在这些任务上的失败案例，是提出修改建议的依据；
- **考试任务集**：用来验证修改效果的任务。这些任务从不展示给提建议的环节——就像模拟考和高考分开出题，防止「针对考题刷分」式的虚假进步。

5.2 三阶段循环

整个闭环由三个阶段组成，全部由同一个（被冻结的）模型完成：

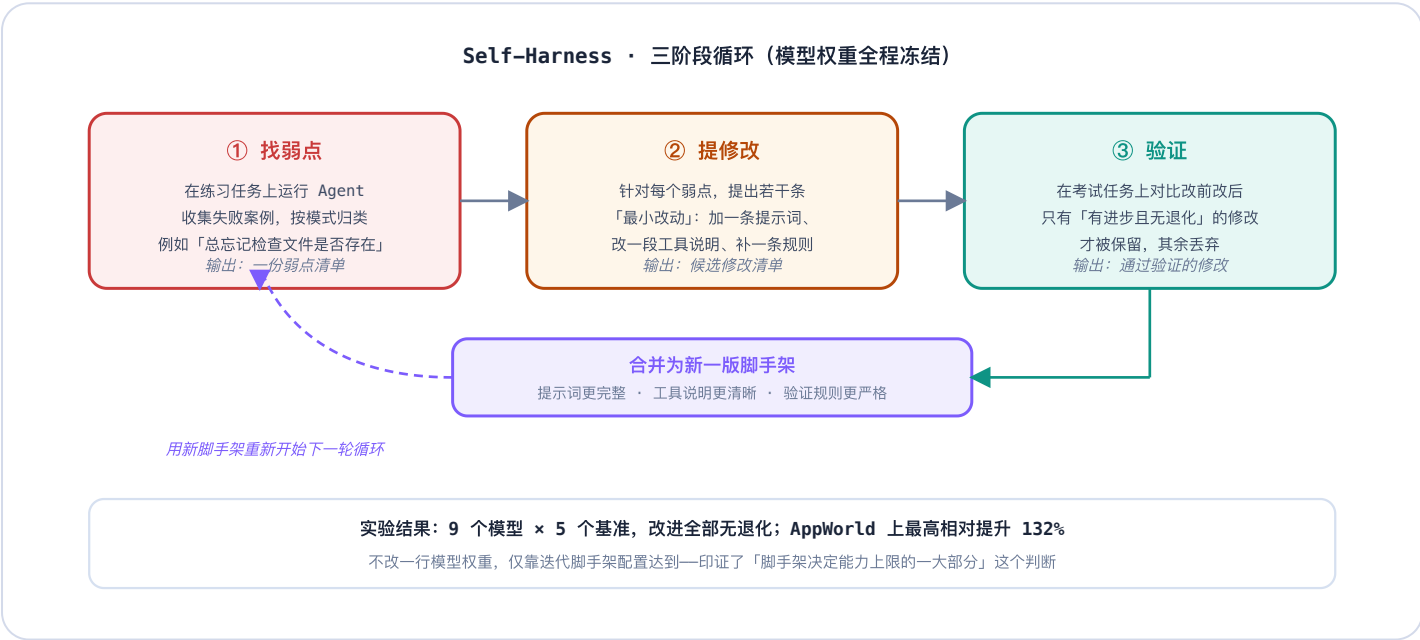


图 5-1 · Self-Harness 三阶段循环：找弱点 → 提最小修改 → 回归验证，通过才合并

「验证」为什么是关键一步

自动修改最大的风险是**自我感觉良好**：模型提出一条修改，很可能在它自己看来都是合理的，但实际效果可能变差，也可能只是「恰好适应了练习任务」。所以第三阶段的两套任务集设计很重要：考试任务从不给提案环节看，验证标准是「有进步且无退化」——把「觉得有用」换成「测过有用」，这是整个闭环里最朴素也最有效的安全措施。

5.3 底座与闭环如何配合

Self-Harness 的论文实验是在自己的环境里跑的。但设想把它放到 dsh 这样的底座上运行，整套流程会衔接得很自然：

阶段① 找弱点

Agent 在任务上的所有行为都记录在会话日志里——失败分析可以直接基于日志回放，不需要额外埋点。

阶段② 提修改

修改的对象是插件或配置。因为一切皆插件，「补一条提示词」「换一个工具实现」「加一层检查」都对应明确的插件操作，可精确到单个零件。

阶段③ 验证与合并

验证通过 → 新插件生效；验证不通过 → 卸载新插件，登记过的撤销方法自动执行，环境回到改动之前。回滚不需要任何额外操作。

整个过程

正在服务的其他会话由日志独立承载，自进化循环可以在不打断任何用户任务的情况下持续进行。

本节要点

底座回答「怎么改才安全」（可替换、可撤销、不打扰运行），闭环回答「改什么、值不值得改」（找弱点、提最小修改、用独立任务集验证）。两者相加，才是一个完整的自进化方案——前者已经在 dsh 中实现，后者是正在被验证的研究方向。

06 经验路线：ACE 论文

Self-Harness 回答了「怎么改」，但改动需要的素材——从哪里发现弱点、归纳经验——同样重要。ACE 展示了一条「规模化经验挖掘」的路线，正是第 1 节所说分析路线的代表性实践。

论文信息

Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models

(arXiv:2510.04618)，斯坦福大学、SambaNova、西蒙弗雷泽大学，2025 年 10 月。核心主张：除了训练权重，语言模型还有一条便宜的改进通道——让上下文自己进化。

6.1 核心思想：改进通道不止权重一条

一个 Agent 的「聪明程度」由两部分共同决定：**模型权重**（靠训练改进）和**上下文**（提示词、记忆、工具说明——靠工程改进）。上下文通道的优势与第 2 节的判断一致：**改动成本低、即时生效、可以撤销**。ACE 在此基础上更进一步：上下文里的经验不必依赖人工编写，可以从 Agent 自己的执行轨迹中**批量挖掘**。

6.2 工作机制：从轨迹到经验的流水线

ACE 的核心组件叫 **Context Rover**——一条由 LLM 驱动的分析流水线，分四步运转：



挖掘出的经验沉淀在三个位置：**结构化记忆**（按需检索的笔记，比如「处理大型仓库时先看目录结构」）、**工具说明**（越用越准的「使用手册」）、**系统提示词**（批量进化）。三者合起来，构成一个不训练也能持续变聪明的 Agent。

6.3 结果与本文主线的呼应

- **效果**：在不做任何模型微调的前提下，把一个开源模型在 SWE-bench Verified（真实软件修复基准）上的通过率提升到 **54.6%**，超过当时所有同类方法；上下文的有效利用率约提升一倍，单任务成本约 0.5 美元。
- **呼应数据门槛 (1.1)**：ACE 的效果建立在大量执行轨迹之上——轨迹的收集与筛选、验证任务的隔离，正是第 1 节说的数据层问题。没有干净的任务数据，经验挖掘同样会「挖偏」。
- **呼应分析路线 (1.2)**：它证明了「不训练、靠归纳」这条路能走多远——比很多人预期的更远；但论文也坦承，归纳质量受限于模型自身的理解能力，离最优解仍有距离。这正好印证了第 1 节的判断：分析路线性价比高，但天花板取决于基座模型。

本节要点

把 Self-Harness 与 ACE 放在一起看，分析路线的两个环节都有了代表工作：**Self-Harness 管决策**（发现弱点 → 提最小修改 → 回归验证），**ACE 管素材**（从大量轨迹中稳定地产出可复用的经验）。两者都绕开了训练权重，也都依赖第 1 节所说的数据基建。

07 总结

回顾全文，DeepSeek 对 Agent 自进化给出了一条清晰的路线。

判断

自进化不等于改模型权重。提示词、工具、流程这些「脚手架」层面的配置，同样决定 Agent 的能力，而且改起来成本低、可解释、可撤销。先把这一层做好，是务实的第一步。

底座

要让脚手架能被安全修改，运行环境本身要先被改造：一切皆插件（结构可替换）、修改登记撤销方法（改动可回滚）、会话写入独立日志（运行不受干扰）。dsh 把这三点做成了工程现实。

闭环与经验

在底座之上，Self-Harness 用「找弱点 → 提最小修改 → 独立任务集验证」决定改什么；ACE 从执行轨迹中批量挖掘可复用经验。两者都表明：不动权重、只迭代脚手架与上下文，就能获得可观且可验证的提升。

这条路线对初学者也有一点启发性的观察：自进化听起来像科幻，但拆开看，每一步都是朴素的工程问题——**怎么存档（撤销方法）、怎么隔离（插件边界）、怎么验货（回归测试）**。把这些基础问题一个个解决掉，复杂的愿景自然就站住了。这大概也是 DeepSeek Harness 最值得参考的地方：它没有发明玄妙的概念，而是把「让系统能承受修改」这件基本的事，认真做到了底。

同时也要诚实地说，这条路并不完整：**数据层**（任务筛选与验证集设计）还没有成熟的方法论；**离线层**的训练路线和分析路线各有硬天花板，「参数化脚手架再训练」这类中间方向仍在探索；**在线层**的可控性方案——包括 DeepSeek Harness 本身——也还不完美。自进化是一个仍在快速演进的方向，本文描述的是当下最有工程说服力的一段路线，而不是终点。

- Cordis: **A Programming Paradigm for Spatiotemporal Composability** (北京大学 + DeepSeek-AI)
- Self-Harness: **arXiv:2606.09498** (上海人工智能实验室, v2 2026-08)
- ACE: **Agentic Context Engineering**, **arXiv:2510.04618** (斯坦福大学、SambaNova、西蒙弗雷泽大学)
- DeepSeek Harness 源码研究与内部报告 (本文架构描述的依据)